

HelixVPN — `helix-ui` Flutter Application Specification

HelixVPN — helix-ui Flutter Application Specification

Revision: 1 **Last modified:** 2026-07-04T12:00:00Z

Status note (added in the production-hardening pass, 2026-07-04). This is the original Claude-authored source document ([04_UI]) that seeded Volume 4 of the final/ specification set. Its architecture-level content (two-cores strategy, melos layout, FFI surface sketch, per-platform shim matrix, design system, Riverpod) has since been absorbed, deepened, and in places corrected by the nano-detail documents under docs/research/mvp/final/v04-client/ (see final/03-client-core-and-ui.md §"Sources verified" for the absorption map) — those documents are authoritative on any point of conflict per final/SPECIFICATION.md's provenance rule. This file is retained as the readable narrative origin and primary citation source ([04_UI §N]); it is **not** independently rewritten in this pass beyond this status note.

Companion to: HelixVPN-Architecture-Refined.md, HelixVPN-Phase0-Spike.md, HelixVPN-Phase1-MVP.md, HelixVPN-Phase2-Parity.md. **Role in the suite:** this is the **third reuse pillar** (architecture §5.5) — the single Dart/Flutter codebase that produces **all three apps** (Access, Connector, Console) across **all eight platforms** (iOS, Android, Aurora, HarmonyOS, Windows, Linux, macOS, Web), sitting on top of the unchanged **Rust helix-core** and the **generated API clients**.

The discipline of this spec: **one widget tree, three flavors, eight targets — and the only per-platform code is the thin tunnel shim**. Everything a user sees is shared; everything the OS demands (the VPN extension lifecycle) is isolated behind a single platform-channel contract.

0. What helix-ui is and is not

It IS	It is NOT
All screens, the design system, navigation, state, the API/WS client, the FFI bindings to helix-core	The tunnel itself (that's Rust helix-core)
One responsive/adaptive layout that reflows phone → tablet → desktop → web	A separate app per form factor
Three build flavors from one lib/	Three codebases
The owner of <i>presentation</i> and <i>orchestration</i>	The owner of crypto, packet I/O, or obfuscation

The brief's app requirements — *shared reusable codebases, small app sizes, small memory, blazing fast, all platforms* — are met by this split: heavy lifting in a tiny Rust core, UI in AOT-compiled Flutter (never Electron), per-platform code reduced to the irreducible tunnel shim.

1. Project structure (monorepo package under helixvpn/helix-ui/)

```
helix-ui/
├─ packages/
│   └─ helix_design/           # design system: tokens, theme,
widgets, icons, motion
│   └─ helix_core_fffi/       # flutter_rust_bridge bindings to
helix-core (Access/Connector)
│   └─ helix_api/             # generated OpenAPI REST client +
WS/SSE client (all apps)
│   └─ helix_domain/         # shared models, state
(Riverpod), use-cases
│   └─ helix_l10n/            # localization (en, ru, zh-Hans,
... – Aurora & HarmonyOS markets)
├─ apps/
│   └─ access/                 # flavor: end-user VPN app (uses
core_fffi + design + domain)
│       └─ lib/main_access.dart
│       └─ <platform shims>/  # ios/ android/ windows/ macos/
linux/ ohos/ aurora/
│   └─ connector/             # flavor: network-side config UI
(uses core_fffi in advertise mode)
│       └─ lib/main_connector.dart
│       └─ console/            # flavor: admin console (uses
helix_api ONLY – no core_fffi)
│           └─ lib/main_console.dart
```

└─ melos.yaml # monorepo task runner (build/
test/lint across packages)

- **melos** manages the multi-package workspace (bootstrap, versioning, run-everywhere scripts).
 - **Access & Connector** depend on `helix_core_ffi` (they drive a tunnel); **Console** does not (it's pure API — and is the only flavor that builds to **Web**, since browsers can't tunnel; architecture §5.7).
 - Platform shim folders live under each app that needs a tunnel; they implement one contract (§6).
-

2. The three flavors from one tree

Flutter `--flavor` + distinct entrypoints select the app; shared code is 80–90% of each.

```
// apps/access/lib/main_access.dart
void main() => runHelixApp(
  flavor: HelixFlavor.access,
  home: const ConnectScreen(), // the big connect button
  capabilities: const {Capability.tunnel, Capability.account,
    Capability.splitTunnel},
);

// apps/connector/lib/main_connector.dart
void main() => runHelixApp(
  flavor: HelixFlavor.connector,
  home: const ConnectorDashboard(), // advertise CIDs, local
    status
  capabilities: const {Capability.tunnel, Capability.advertise,
    Capability.localAcl},
);

// apps/console/lib/main_console.dart
void main() => runHelixApp(
  flavor: HelixFlavor.console,
  home: const ConsoleShell(), // tenants/devices/networks/
    policy/audit
  capabilities: const {Capability.admin}, // no tunnel
    capability → no core_ffi
);
```

`runHelixApp` (in `helix_domain`) wires the theme, router, providers, and the capability set that gates which features compile/appear. A capability the flavor lacks is tree-shaken out, keeping each binary lean.

3. Design system (helix_design)

A VPN client's UI has one emotional center of gravity: **am I protected right now?** The design system is built around making that state instantly legible, then getting out of the way. Material 3 is the substrate; brand tokens override the defaults so it never reads as a stock Flutter app (a real risk to design against).

3.1 Design tokens (the single source of truth)

```
// helix_design/lib/tokens.dart
class HelixTokens {
  // ---- connection-state palette (the signature semantic colors)
  ----
  static const stateDisconnected = Color(0xFF6B7280); // neutral
  grey
  static const stateConnecting   = Color(0xFFFF59E0B); // amber,
  in-motion
  static const stateConnected    = Color(0xFF10B981); // green,
  safe
  static const stateDanger       = Color(0xFFEF4444); // leak /
  kill-switch tripped

  // ---- brand seed (drives Material 3 ColorScheme, then
  overridden per role) ----
  static const brandSeed = Color(0xFF3B5BDB);

  // ---- spacing scale (4pt base) ---- 4,8,12,16,24,32,48
  // ---- radius scale ---- sm 8, md 12, lg 20, pill
  999
  // ---- typography ---- display / headline / title / body /
  label (M3 type scale, brand font)
  // ---- motion ---- fast 120ms, base 220ms, slow 360ms;
  emphasized easing for state changes
  // ---- elevation ---- 0..3, used sparingly; flat surfaces,
  color-as-hierarchy
}
```

- **Light + dark** both first-class (privacy users skew dark; respect system setting + manual override).
- Tokens are consumed via a HelixTheme extension on ThemeData, so screens never hardcode colors/spacing — they reference roles (context.helix.stateConnected), which keeps the eight platforms visually identical and re-themeable.

3.2 Signature components

Component	Purpose	Notes
ConnectButton	the giant tap target; morphs disconnected→connecting→connected	color + label + subtle

Component	Purpose	Notes
StatusChip	current transport + path + RTT (MASQUE · direct · 23ms)	motion driven by the FFI status stream (§5) live; the proof that real-time works
ExitPicker	choose exit gateway / network / multi-hop chain	searchable, RTT-sorted, jurisdiction labels for multi-hop
ShieldIndicator	kill-switch / DNS-protection / DAITA / PQ badges	each a small affordance with an honest cost note on tap
NetworkTile	a joined network (Console/Access)	shows connector health, advertised CIDRs, your access level
PolicyEditor	Console ACL editing	text + the effect-diff preview (Phase 2 §7.2)
AdaptiveScaffold	responsive shell: BottomNav (phone) ⇌ NavigationRail (tablet/desktop) ⇌ extended rail (web)	one widget, all form factors (§7)

3.3 Avoiding the "default Flutter" look

Concrete moves: a real brand typeface (not Roboto default), a restrained flat surface treatment with color (not elevation) for hierarchy, one confident accent, generous spacing, and **motion reserved for state change** (the connect transition is the one place to spend animation budget). The connection state owns the screen; settings recede.

4. State management & data layer

Riverpod is the choice: compile-safe dependency injection, first-class support for turning the FFI/WS **streams** into reactive UI, trivial to test (override providers), and no BuildContext coupling. (Bloc is the viable alternative if the team prefers explicit event/state classes; the architecture below maps cleanly onto either.)

4.1 Layering

UI (widgets) $\rightarrow$ Providers (state) $\rightarrow$ Use-cases
 $\rightarrow$ Repositories

└ helix_core_ffl (tunnel control + status stream)

└ helix_api (REST: enroll, devices, policy, networks)

└ WS/SSE client (live events for Console)

4.2 The tunnel state provider (Access/Connector)

```
// helix_domain: turn the Rust core's event stream into reactive
// Dart state
final tunnelStatusProvider = StreamProvider<TunnelStatus>((ref) {
  final core = ref.watch(helixCoreProvider);
  return core.statusStream(); // flutter_rust_bridge
  // StreamSink → Dart Stream
});

final connectControllerProvider =
  AsyncNotifierProvider<ConnectController,
    void>(ConnectController.new);

class ConnectController extends AsyncNotifier<void> {
  Future<void> connect({String transport = 'auto'}) =>
    ref.read(helixCoreProvider).start(transport: transport);
  Future<void> disconnect() => ref.read(helixCoreProvider).stop();
}

// the ConnectButton just reflects the stream – no polling, no
// manual refresh
final status = ref.watch(tunnelStatusProvider);
ConnectButton(
  state: status.valueOrNull ?? TunnelStatus.down(),
  onTap: () => ref.read(connectControllerProvider.notifier)
    .toggle(status.valueOrNull),
);
```

This is the architecture's "real-time, event-driven, the whole system" promise expressed at the UI layer: the button is a pure function of a stream the Rust core pushes.

4.3 Console live data

The Console subscribes to the control-plane WS/SSE (GET /v1/stream, Phase 1 §8) and folds events (device.online, route.changed, policy.compiled) into Riverpod state, so device lists, the topology view, and the audit feed update live without refresh.

4.4 Offline / optimistic behavior

Access/Connector keep last-known status and a local intent (user wants connected) so a flaky control channel never makes the UI lie; the core's status stream is always the source of truth for *actual* protection state, and the UI distinguishes "intended" from "actual" (e.g., Reconnecting..).

5. FFI integration — Flutter $\rightleftharpoons$ helix-core (helix_core_ffi)

The Phase 0 G5 boundary, now the production contract. **flutter_rust_bridge v2** generates the Dart $\rightleftharpoons$ Rust glue from the helix-ffi Rust surface (architecture §9 / Phase 0 §9).

```
// helix_core_ffi public surface (generated + thin wrapper)
abstract class HelixCore {
  Future<void> start({required String transport, String?
    mapPathOrSession});
  Future<void> stop();
  Stream<TunnelStatus> statusStream();           // Connecting/
    Handshaking/Connected{transport,path,rtt}/...
  Future<List<ExitOption>> exits();
  Future<void> setExit(String id, {List<String>? multiHopChain});
  Future<void> setShields({bool killSwitch, bool dnsProtection,
    bool daita, bool postQuantum});
  // connector mode:
  Future<AdvertiseResult> advertise(List<String> cidrs);
}
```

Critical division of labor: helix_core_ffi covers *logic and status*. It does **not** own the OS tunnel lifecycle — that's the platform shim (§6), because on most platforms the VPN runs in a separate OS-managed process/extension. The FFI and the shim meet through a single platform-channel contract.

6. Per-platform tunnel shims (the only platform-specific code)

Every shim implements **one contract** and does **only** three things: configure the OS tunnel, hand packets to/from helix-core, and report lifecycle. Everything else is shared Dart.

```
// the single contract every platform implements (MethodChannel +
// EventChannel)
abstract class TunnelPlatform {
  Future<void> startTunnel(TunnelConfig cfg); // OS asks
// permission, sets up TUN
  Future<void> stopTunnel();
  Stream<PlatformTunnelEvent> events(); // up/down/
// permissionDenied/revoked
}
```

Platform	OS mechanism	Shim language	How helix-core is loaded
iOS / macOS	NEPacketTunnelProvider (Network Extension)	Swift	Rust staticlib linked into the extension; packetFlow ⇄ core
Android	VpnService + foreground service	Kotlin	core via JNI; ParcelFileDescriptor fd ⇄ core
Windows	wireguard-nt/wintun + privileged service	C#/Rust	service hosts core; Flutter↔service over named-pipe IPC
Linux	kernel WG or tun	Rust/Dart FFI	core in-process (desktop) or a small helper daemon
HarmonyOS NEXT	Network Kit VPN extension ability	ArkTS shim → NAPI	core as a native .so via N-API; ArkTS MethodChannel bridges to Flutter (OpenHarmony SIG fork)
Aurora OS	Qt/C++ network backend + tun	C++ shim	core linked as C lib into the Qt backend; Friflex Flutter plugin bridges (OMP fork)
Web (Console only)	— none —	n/a	no tunnel; Console is API-only

The payoff: the `ConnectButton`, `ExitPicker`, settings, account, policy views — every pixel — are identical Dart across all of these. Adding HarmonyOS or Aurora is "write one shim that satisfies `TunnelPlatform` + bend the build," not "port the app."

6.1 HarmonyOS & Aurora specifics (the Phase 3 reach work)

- **HarmonyOS:** build with the OpenHarmony SIG Flutter fork (ohos channel → HAP). The VPN ability and any platform plugins are written in **ArkTS** and bridge via `MethodChannel/NAPI` to the Rust `.so`. Sign in DevEco. Fork lags mainline Flutter — pin versions and budget plugin work; the *UI* ports for free, the *shim* does not.
 - **Aurora:** build with the OMP Russia Flutter fork (`flutter-aurora` → signed RPM). Tunnel backend in Qt/C++; Flutter plugins via Friflex. Toolchain is Russian-hosted (GitLab `omprussia` / Mos.Hub) and Aurora is enterprise/government-oriented — treat as an enterprise SKU with its own CI runners and signing, per architecture §5.7.
-

7. Responsive & adaptive layout (one tree, every form factor)

A single `AdaptiveScaffold` reflows by width breakpoint — the convergent approach that lets the same Access app feel native on a phone and a desktop, and the Console feel native on web and desktop.

// breakpoints

compact (< 600) → `BottomNavigationBar`, single pane, full-width `ConnectButton`

medium (600–1024) → `NavigationRail`, master/detail where useful

expanded (> 1024) → extended `NavigationRail` + multi-pane

(Console: list | detail | live events)

- Use `LayoutBuilder/MediaQuery` breakpoints (or `flutter_adaptive_scaffold`); never branch on `Platform.isX` for layout — branch on **size**, so a desktop window resized small behaves like a phone and web "just works" responsively (the brief's "fully responsive web").
 - Input adaptivity: pointer + keyboard shortcuts on desktop/web (e.g., ⌘K command palette in Console), touch targets ≥ 48dp on mobile.
 - The Console's heavy admin tables, topology graph, and policy editor are **deferred-loaded** so they never bloat the Access/Connector binaries (they're different flavors anyway, but shared widgets stay tree-shakeable).
-

8. Key screens by flavor

8.1 Helix Access (end user)

- **Connect** — the `ConnectButton` + `StatusChip` (transport · path · RTT); one tap to protection.
- **Exits / Networks** — `ExitPicker`: privacy exits (by country/RTT) and joined networks you're authorized to reach; multi-hop chain builder (entry/exit, jurisdiction labels).
- **Shields** — kill-switch, DNS-leak protection, DAITA ("maximum privacy", honest cost), post-quantum toggle, split-tunnel (per-app/route).
- **Account** — managed (OIDC) or anonymous device-token; device list + revoke; no PII in anonymous mode.

8.2 Helix Connector (network operator)

- **Dashboard** — connector health, current transport/path to gateway, throughput counters (aggregate, no logs).
- **Advertise** — add/remove CIDRs (`advertise()` → `AdvertisePrefixes`); shows conflicts (overlapping-CIDR warnings from `route.conflict.detected`).
- **Local ACL** — optional site-local restrictions layered under tenant policy.
- **Headless mode** — the same core runs as a daemon with no UI; this flavor's UI is the *optional* config surface (architecture §5.4).

8.3 Helix Console (admin) — web + desktop

- **Devices** — live list (online/offline via WS), enroll-token minting (QR), revoke.
 - **Networks** — connectors, advertised prefixes, health, overlapping-CIDR resolution UX (hides the 4via6 mechanics).
 - **Policy** — `PolicyEditor` with the **effect-diff** preview before apply; version history + one-click rollback.
 - **Audit** — live control-action feed (never traffic).
 - **Topology** — live graph of who-reaches-what, updated from the event stream.
-

9. Localization, accessibility, theming

- **i18n/l10n** (`helix_l10n`, Flutter intl/ARB): English baseline, **Russian** (Aurora market) and **Simplified Chinese** (HarmonyOS market) as first-tier, RTL-ready. Censorship-region users are core users — localized, clear connection state matters more here than anywhere.

- **Accessibility:** semantic labels on the `ConnectButton/`
`ShieldIndicator` (state must be announced, not just colored —
never rely on color alone for protected/unprotected), large-text
and high-contrast support, full keyboard nav on desktop/web.
- **Theming:** light/dark + system; tokens (§3.1) make a future
white-label/self-host rebrand a token swap, not a refactor.

10. Size, memory & performance budget

The brief demands tiny, fast, stable. Concrete targets and the means:

Budget	Target	How
Mobile install size	Access < ~15- 20 MB per ABI	--split-per-abi, tree-shake icons/ fonts, no bundled video, lean Rust staticlib (LTO+strip)
Cold start	< 1 s mid- range mobile	AOT, deferred non-critical providers, no work on the UI thread at launch
Idle memory (app)	single- digit→low-tens MB	Flutter AOT (no Electron); tunnel core memory is the Rust budget (Phase 0 §6)
Frame budget	60/120fps, no jank	const widgets, <code>RepaintBoundary</code> on animated state, motion only on state change
Web (Console)	fast first paint	Wasm/CanvasKit, route-level deferred loading, no <code>core_ffi</code> weight

Non-negotiable: never Electron, never a webview-wrapped app
for the native targets. Flutter AOT + Rust core is the entire
reason the size/speed targets are reachable on eight platforms at
once.

11. Build, flavors & CI

- **melos** runs `analyze/test/build` across packages; each app builds
per platform via its flavor.
- **CI matrix:** mainline Flutter for iOS/Android/Win/macOS/Linux/
Web; **separate runners** for the OpenHarmony SIG fork
(HarmonyOS HAP, DevEco signing) and the OMP fork (Aurora
RPM signing) — these are pinned to specific fork versions and
isolated so a fork lag never blocks mainline releases.
- **Codegen in CI:** `flutter_rust_bridge` bindings, OpenAPI→Dart
client, and intl ARBs are generated and checked for drift —

the apps can never silently diverge from the Rust core or the control-plane API (architecture §4.2).

- **Signing/notarization** per platform (Apple notarize, Windows driver/service sign, HarmonyOS DevEco, Aurora RPM).

12. Testing

Layer	Tooling	What
Widget tests	<code>flutter_test</code>	each component renders per state (ConnectButton across all TunnelStatus)
Golden tests	<code>golden_toolkit</code>	the design system stays visually stable across themes/breakpoints
State tests	Riverpod overrides	controllers/use-cases with a fake HelixCore + fake API
Integration	<code>integration_test</code>	flavor flows: connect→status, advertise→accepted, policy edit→effect-diff
FFI contract	shared fixtures	the Dart TunnelStatus/ExitOption models match the Rust helix-ffi types (generated, asserted)
Platform shim	per-OS smoke	tunnel up/down/permission/revoke on real devices (the shim is the only untyped seam)

Fake the core and API with the *same model types* the generators produce, so tests exercise real contracts.

13. How helix-ui ties the suite together

helix-ui is the visible face of everything the other four documents build:

- It consumes the **Rust helix-core** (Phase 0) through the FFI proven in G5 — UI is a pure function of the core's status stream.
- It speaks to the **Go control plane** (Phase 1) via the generated REST/WS client; the Console's live views are the Phase 1 event taxonomy made visible.
- It exposes **Phase 2 features** (multi-hop picker, DAITA/PQ shields, direct-vs-relay path indicator, GitOps effect-diff) as first-class UI without new architecture — they were always just fields on the map and the API.

- It delivers the brief's hardest app requirement — **eight platforms, one codebase, tiny and fast** — by confining platform code to a single TunnelPlatform shim and keeping every pixel shared.

This completes the HelixVPN specification suite: **Architecture (what) → Phase 0 (prove) → Phase 1 (MVP) → Phase 2 (parity + reach) → helix-ui (the app layer across all of it)**. Five documents, one coherent system, every interface drawn so the next layer is additive.

End of helix-ui Flutter application specification.